Halving High-Severity Security Findings in REST APIs

OWASP Top 10 Hardening: Input Validation, Rate Limiting, Idempotency, OAuth2/JWT, Secrets Management and CI Security Gates

A simple-language guide to the project: summary, components, workflow and architecture

> **The project in one sentence (as written)**
> Reduced high-severity security findings 50% by hardening REST APIs against OWASP Top 10 risks with input validation, rate limiting, idempotency, OAuth2/JWT authentication and authorization, secrets management, and SAST and dependency scanning gates in CI.

Contents

1. Overall summary (in simple terms)
2. Decoding the project sentence, phrase by phrase
3. Key ideas explained simply (a mini-glossary)
4. Architecture diagrams and a full walk-through of them
5. Components explained in detail
6. Workflows explained end to end
7. How the controls map to the OWASP Top 10
8. Where the 50% came from
9. Quick recap

Note: the project sentence is a short summary. This guide describes a realistic, typical design for such a project so that every part of the sentence can be explained concretely. Component names, limits, timings and finding counts are illustrative examples chosen for this guide, not data from a specific system.

1. Overall Summary (in Simple Terms)

A **REST API** is the set of "doors" through which apps talk to a company's servers: `GET /orders/42` to read an order, `POST /payments` to make a payment, and so on. Think of a large **office building** with many doors. Every door must check who is coming in, what they are allowed to do, and whether what they are carrying is safe.

Security reviews (penetration tests, automated scanners, bug reports) kept finding **high-severity problems** in these APIs: doors that let the wrong people into the wrong rooms, forms that accepted dangerous input, passwords left lying around in code, and old software parts with known holes. The **OWASP Top 10** is the industry's well-known list of the most common and dangerous web security risks, and most findings matched items on it.

This project fixed the root causes in two places: in the **running API** (every request now passes a line of security checks) and in the **build pipeline** (code with serious problems can no longer reach production). As a result, the number of high-severity findings dropped by **about half** (for example, from 40 to 20 over comparable review periods).

What was done	Building analogy	What it stops
OAuth2 / JWT authentication	A guard checks a tamper-proof, expiring badge at the door.	Strangers and forged or expired identities getting in.
Authorization	The badge only opens the rooms you are allowed into, and only your own filing cabinet.	Users reading or changing other people's data.
Input validation	Every form must be filled in exactly as the template says, or it is rejected.	Injection attacks, crashes and data corruption from bad input.
Rate limiting	A turnstile that only lets each person through so many times per minute.	Password guessing, scraping and overload attacks.
Idempotency	A receipt number: handing in the same payment slip twice does not pay twice.	Double charges and duplicate orders from retries or replays.
Secrets management	Keys kept in a locked safe that changes its locks regularly, not under the doormat.	Leaked passwords and API keys being found in code or logs.
SAST and dependency scanning gates in CI	A building inspector checks the plans and the materials before anything is built.	New vulnerable code and known-vulnerable libraries reaching production.

In one breath

Every API request now has to pass a fixed line of checks (rate limit → who are you → what may you do → is your input valid → have we already done this) before any business logic runs; secrets live in a vault instead of in code; and the CI pipeline refuses to ship code with serious findings. Fewer holes get in, and old ones were closed: high-severity findings fell by half.

2. Decoding the Project Sentence, Phrase by Phrase

Phrase	What it means in simple words
Reduced high-severity security findings 50%	Half as many serious security problems were found in reviews, scans and tests than before (same scope, comparable period).
High-severity	Problems rated High or Critical, usually by a CVSS score of 7.0 or more: easy to exploit and/or very damaging.
Hardening	Making something harder to attack by removing weaknesses and adding defences.
REST APIs	Web interfaces where apps use HTTP methods (GET, POST, PUT, DELETE) on URLs to read and change data, usually in JSON.
OWASP Top 10 risks	The best-known list of the ten most important web application security risk categories, published by the OWASP foundation.
Input validation	Checking every piece of incoming data against strict rules (type, length, format, allowed values) and rejecting anything else.
Rate limiting	Capping how many requests a client may make in a time window.
Idempotency	Making sure that repeating the same request (for example after a network retry) has the same effect as doing it once.
OAuth2	The standard way for an app to obtain an access token from a trusted login service instead of handling passwords itself.
JWT	JSON Web Token: a signed, tamper-evident token that carries who the user is, what they may do, and when it expires.
Authentication and authorization	Authentication = proving who you are. Authorization = deciding what you are allowed to do.
Secrets management	Storing passwords, keys and tokens in a secure vault, giving them only to services that need them, and rotating them.
SAST	Static Application Security Testing: automatically scanning source code for insecure patterns without running it.
Dependency scanning	Checking the third-party libraries the code uses against databases of known vulnerabilities (CVEs). Also called SCA.
Gates in CI	Automatic checks in the Continuous Integration pipeline that block a change from merging or deploying if serious problems are found.

3. Key Ideas Explained Simply (a Mini-Glossary)

3.1 REST APIs and endpoints

- **Endpoint:** a URL plus a method, such as `GET /orders/{id}`.
- **Request:** method, URL, headers (including the `Authorization` header with the token) and an optional JSON body.
- **Response:** a status code (200 OK, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 Server Error) plus a body.

3.2 OWASP Top 10 and severity

OWASP publishes two lists that matter here: the general **OWASP Top 10** for web applications and the **OWASP API Security Top 10**, which focuses on API-specific problems such as broken object-level authorization. Each security finding is rated for severity, commonly with **CVSS**: Low, Medium, High (7.0–8.9) and Critical (9.0–10). This project focused on High and Critical.

3.3 Authentication versus authorization

- **Authentication (AuthN):** "Who are you?" Proven by a valid token from the identity provider.
- **Authorization (AuthZ):** "Are you allowed to do this, to this particular thing?" For example, a user may read orders (scope check) but only **their own** orders (object-level check).
- **BOLA / IDOR:** the most common serious API flaw. The API checks that you are logged in, but not that order 42 is yours, so changing the number in the URL shows someone else's data.

3.4 OAuth2 and JWT

- **Identity provider (authorization server):** the trusted login service that checks passwords and multi-factor authentication and issues tokens.
- **Authorization code flow with PKCE:** how user-facing apps get tokens safely: the user logs in at the identity provider, the app receives a one-time code and exchanges it for tokens. PKCE stops stolen codes being reused.
- **Client credentials flow:** how one backend service gets a token to call another, with no user involved.
- **Access token:** short-lived (for example 10–15 minutes); sent with each API call.
- **Refresh token:** longer-lived; used only with the identity provider to get new access tokens; can be revoked.
- **JWT parts:** header (algorithm, key ID), payload (claims) and signature. Anyone can read a JWT, but nobody can change it without breaking the signature.
- **Important claims:** `iss` (who issued it), `aud` (which API it is for), `sub` (the user), `exp` (expiry), `scope` (permissions).
- **JWKS:** the identity provider's published public keys, used by the API to check signatures.

3.5 Input validation

- **Allowlist, not blocklist:** define exactly what is allowed (an integer from 1 to 100; an email format; one of three status values) and reject everything else. Trying to list every bad input always misses something.
- **Schema validation:** the API's OpenAPI contract describes every field; requests are checked against it automatically.
- **Injection:** when input is treated as code (SQL, shell commands). Validation reduces it; **parameterized queries** eliminate it at the database layer.
- **Mass assignment:** a client sneaks in extra fields like `"role": "admin"`. Rejecting unknown fields stops it.

3.6 Rate limiting

- **Token bucket:** each client has a bucket that refills at a steady rate; each request takes a token; an empty bucket means "wait".
- **Sliding window:** count requests in the last N seconds.
- **Layers:** a global limit per IP, a per-client limit, and stricter limits on sensitive endpoints (login, password reset, payments).

3.7 Idempotency

Networks fail. If a phone sends "pay $50" and the connection drops before the answer arrives, the app retries. Without idempotency the customer may pay twice. With an **Idempotency-Key** header (a unique ID chosen by the client per operation), the server remembers the first result and returns the same result for repeats, without doing the action again.

3.8 Secrets management

- **Secret:** anything that grants access: database passwords, API keys, signing keys, client secrets.
- **Vault / secrets manager:** encrypted storage with access control and audit logs (for example HashiCorp Vault or a cloud secrets manager).
- **Rotation:** changing secrets regularly so a leaked one stops working soon.
- **Dynamic secrets:** credentials created on demand with a short life, unique per service instance.

3.9 SAST, SCA, secret scanning and DAST

- **SAST:** reads source code to find risky patterns such as string-built SQL, disabled certificate checks or unsafe deserialization.
- **Dependency scanning (SCA):** lists every library and version the code uses (a Software Bill of Materials, SBOM) and compares it with known vulnerability databases.
- **Secret scanning:** finds keys, tokens and passwords accidentally committed to code.
- **DAST:** attacks the running API in a test environment to find problems that only appear at runtime.

- **Shift left:** catching problems as early as possible (in the developer's pull request) where they are cheapest to fix.

4. Architecture Diagrams and Full Walk-Through

There are two diagrams. **Figure 1** shows the running system: the line of checks every API request passes and the security services those checks use. **Figure 2** shows the CI/CD pipeline with its security gates, which decides what code is allowed to run in the first place.

How to read Figure 1	Meaning
Blue zone (left)	The runtime request path inside the API platform. Stages 1 to 7 run in this order for every request.
Red boxes	The security controls added or hardened by this project.
Orange zone (right), purple boxes	Supporting services the checks rely on: logs, counters, keys, permission rules, schema, idempotency store, secrets.
Green boxes	Outside systems: clients, the identity provider, the database and downstream APIs, and the CI/CD pipeline.
Dark blue arrows 1–8	The path of a request that passes every check.
Green arrows T and CI	Getting a token from the identity provider; deploying only scanned code.
Purple arrows A–F, K, M	Each check consulting its supporting service; public keys published by the identity provider; logs.

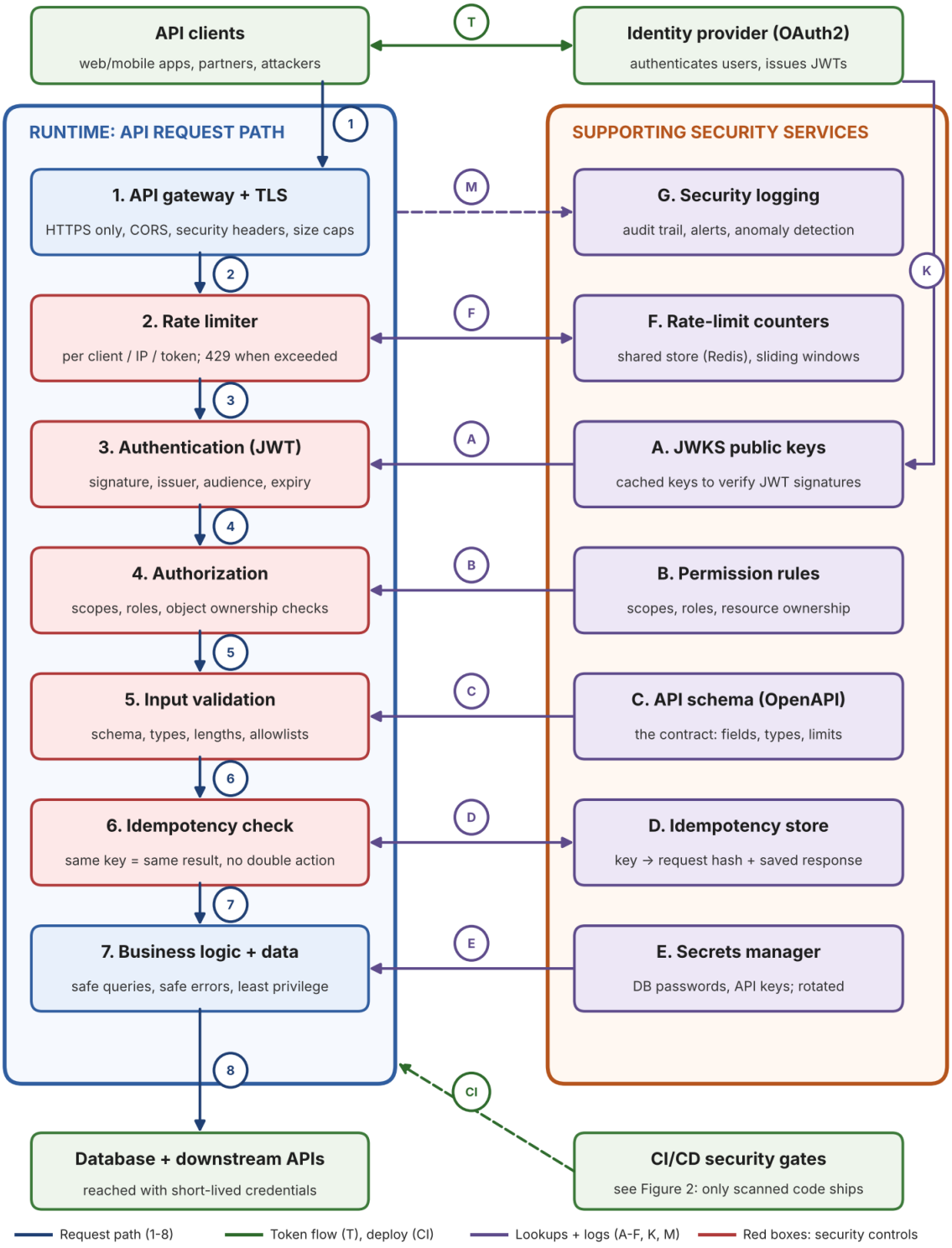


Figure 1. Runtime architecture: the ordered security checks on every API request and the services they use

4.1 The zones of Figure 1

Blue zone – runtime API request path. Usually implemented as an API gateway plus shared middleware in each service, so every endpoint gets the same protection automatically instead of each developer re-implementing it. The order is deliberate: cheap checks that stop floods come first, identity comes before permissions, permissions come before reading the body in detail, and only fully checked requests reach business logic.

Orange zone – supporting security services. These do not receive traffic from clients. They provide what the checks need: counters, keys, rules, schemas, stored results and secrets, plus the security log that records every decision.

Green boxes – outside systems. Clients obtain tokens from the identity provider (arrow T). The API talks to the database and other services using secrets from the manager. The CI/CD pipeline (Figure 2) is the only way code reaches this runtime.

4.2 Every box in Figure 1

Box	Zone	One-line explanation
API clients	External	Web and mobile apps, partner systems, and attackers. All input is treated as untrusted.
Identity provider (OAuth2)	External	Logs users in and issues signed access tokens (JWTs).
1. API gateway + TLS	Runtime	HTTPS only, strict CORS, security headers, request size limits, routing.
2. Rate limiter	Runtime (control)	Limits requests per client, IP and token; answers 429 when exceeded.
3. Authentication (JWT)	Runtime (control)	Verifies the token's signature, issuer, audience and expiry.
4. Authorization	Runtime (control)	Checks scopes and roles, then checks that the specific record belongs to (or is shared with) the caller.
5. Input validation	Runtime (control)	Checks every field against the OpenAPI schema; rejects unknown fields and bad values.
6. Idempotency check	Runtime (control)	For create and payment operations, ensures repeats with the same key do not repeat the action.

Box	Zone	One-line explanation
7. Business logic + data	Runtime	The actual work, using parameterized queries, least-privilege credentials and safe error messages.
G. Security logging	Support	Records every security decision for audit, alerts and investigation.
F. Rate-limit counters	Support	A shared fast store (such as Redis) holding counters so all API instances enforce the same limits.
A. JWKS public keys	Support	The identity provider's public keys, cached, used to verify token signatures.
B. Permission rules	Support	Which scopes and roles allow which actions, plus ownership rules for records.
C. API schema (OpenAPI)	Support	The machine-readable contract for every endpoint's inputs.
D. Idempotency store	Support	Remembers each idempotency key with a fingerprint of the request and the saved response.
E. Secrets manager	Support	Delivers database passwords and API keys at runtime; rotates them.
Database + downstream APIs	External	Where data lives; reached with short-lived credentials.
CI/CD security gates	External	The pipeline in Figure 2; nothing reaches the runtime without passing it.

4.3 Every arrow in Figure 1

Arrow	From → To	What travels along it
T	Clients ↔ Identity provider	The client logs in (OAuth2) and receives an access token and, for user apps, a refresh token.
K	Identity provider → JWKS keys	The provider publishes its public signing keys; the API caches them and refreshes periodically or when it sees a new key ID.
1	Clients → Gateway	An HTTPS request carrying `Authorization: Bearer <JWT>`.

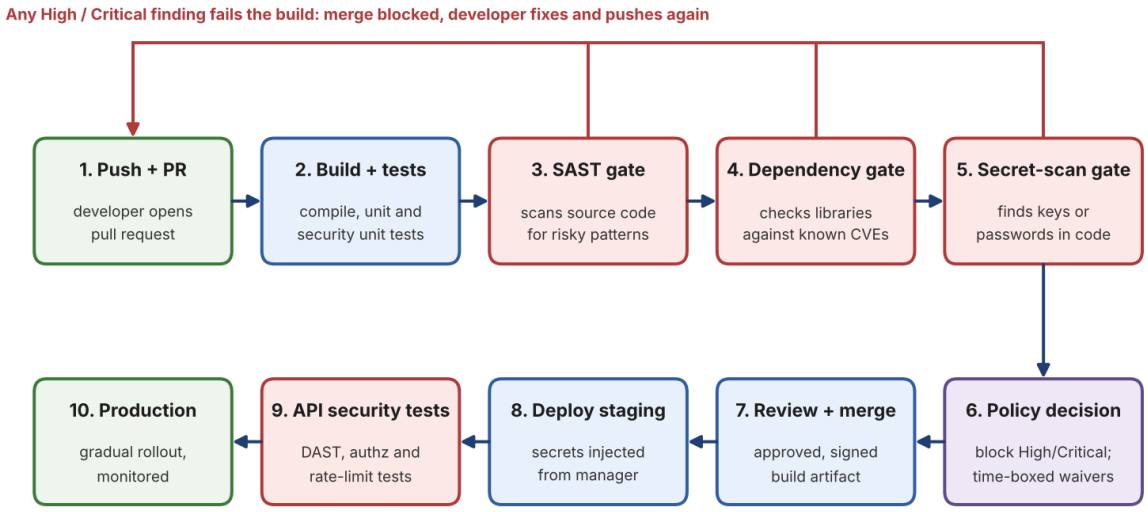
Arrow	From → To	What travels along it
M	Gateway → Security logging	Structured security events from every stage (dashed: telemetry, not traffic).
2	Gateway → Rate limiter	The request, identified by client ID, IP and later the token subject.
F	Rate limiter ↔ Counters	Increment and read the counter for this client and window.
3	Rate limiter → Authentication	Requests within limits.
A	JWKS keys → Authentication	The public key matching the token's key ID.
4	Authentication → Authorization	The verified identity: user or client ID, scopes, roles.
B	Permission rules → Authorization	Rules for this endpoint and record type.
5	Authorization → Validation	Permitted requests.
C	Schema → Validation	The expected shape of this endpoint's input.
6	Validation → Idempotency	Requests whose input is valid.
D	Idempotency ↔ Store	Look up the key; reserve it; later save the response.
7	Idempotency → Business logic	New, fully checked operations.
E	Secrets manager → Business logic	Database and downstream credentials, fetched at start-up and on rotation, never stored in code.
8	Business logic → Database / APIs	Parameterized queries and authenticated calls with least privilege.
CI	CI/CD gates → Runtime	Deployment of approved, scanned builds only.

4.4 Why the checks are in this order

- **Rate limiting first:** it is the cheapest check and protects everything after it, including the token verification work, from floods and brute force.
- **Authentication before authorization:** you must know who someone is before deciding what they may do.
- **Authorization before detailed validation:** unauthorised callers get a plain 401/403 and learn nothing about the internal field rules. (Basic structural checks, such as size limits, already happened at the gateway.)

- **Validation before idempotency:** invalid requests are rejected before a key is reserved, so they cannot block a later valid retry.
- **Idempotency right before business logic:** it is the last gate before something irreversible happens, such as charging money.
- **Object-level checks also inside business logic:** queries always include the owner condition (for example `WHERE id = ? AND owner_id = ?`), so even a mistake in the earlier check cannot leak other users' data.

4.5 Figure 2: the CI/CD pipeline with security gates



Order of the pipeline: top row left to right, then bottom row right to left

Figure 2. CI/CD pipeline: code must pass SAST, dependency and secret-scan gates and a policy decision before deployment

Box	One-line explanation
1. Push + PR	A developer pushes code and opens a pull request; nothing merges directly to the main branch.
2. Build + tests	Compiles the code and runs unit tests, including security unit tests (for example “user A cannot read user B’s order”).
3. SAST gate	Scans the source code for insecure patterns.
4. Dependency gate	Checks every third-party library and version against known vulnerability databases; produces an SBOM.
5. Secret-scan gate	Looks for passwords, keys and tokens accidentally added to code or config.
6. Policy decision	Applies the rule: any High or Critical finding fails the pipeline unless an approved, time-limited waiver exists.
7. Review + merge	Human code review; the merged build is packaged as a signed artifact.
8. Deploy staging	The artifact runs in a test environment, receiving secrets from the secrets manager.
9. API security tests	DAST and automated tests for authorization, validation and rate limits against the running API.

Box	One-line explanation
10. Production	Gradual rollout with monitoring; the same signed artifact that passed staging.
Red feedback line	Any gate failure stops the pipeline and sends results back to the developer in the pull request.

5. Components Explained in Detail

Each component has a fact table (including what it protects against) followed by a step-by-step description.

5.1 External systems

API clients

What it is	Anything that calls the API: the company's web and mobile apps, partner integrations, internal services, and attackers.
Simple analogy	Everyone who walks up to the building's doors.
Design rule	Never trust anything the client sends: IDs, prices, roles, headers, or even the shape of the JSON.

How it works, step by step

1. Obtains a token from the identity provider (arrow T).
2. Sends requests over HTTPS with the token in the `Authorization` header.
3. For create or payment operations, generates a unique `Idempotency-Key` and reuses it on retries.
4. Handles 401 (get a new token), 403 (not allowed), 400 (fix input), 409 (conflict) and 429 (wait and retry with back-off).

Identity provider (OAuth2 authorization server)

What it is	The central service that logs users in and issues tokens. Examples include Keycloak, Okta, Auth0, Azure AD or a cloud provider's identity service.
Simple analogy	The badge office that checks your ID and prints a tamper-proof, expiring badge.
Problem before	Some services checked passwords themselves or used long-lived shared API keys; some tokens never expired.
Change made	All APIs now trust only tokens from this provider. User apps use the authorization code flow with PKCE; services use client credentials. Access tokens last minutes; refresh tokens are rotated and revocable; MFA for sensitive accounts.

How it works, step by step

1. Authenticates the user (password, MFA) or the client service (client secret or certificate).
2. Issues a signed JWT access token with `iss`, `aud`, `sub`, `exp` and `scope` claims.
3. Publishes its public keys at a JWKS endpoint (arrow K) and rotates signing keys on a schedule.

4. Revokes refresh tokens when users log out or accounts are disabled.

5.2 Runtime request path

1. API gateway + TLS

What it is	The single entry point in front of all API services.
Simple analogy	The building's only public entrance.
Protects against	Eavesdropping, cross-site abuse from browsers, oversized payloads, and accidental exposure of internal endpoints (Security Misconfiguration).
Settings	HTTPS only with modern TLS and HSTS; strict CORS allowlist of origins; security headers; maximum body and header sizes; only published routes exposed; internal/admin routes not reachable from the internet.

How it works, step by step

1. Terminates TLS and rejects plain HTTP.
2. Checks request size and basic format; rejects oversized requests (413).
3. Matches the route; unknown routes return 404 without revealing internals.
4. Adds a request ID used in all logs (arrow M).
5. Passes the request to the rate limiter (arrow 2).

2. Rate limiter

What it is	Middleware that counts requests and blocks clients that exceed their limits.
Simple analogy	A turnstile that lets each person through only so often.
Protects against	Brute-force login and token guessing, credential stuffing, scraping, and denial of service (Unrestricted Resource Consumption in the API Top 10).
Example limits	Per IP: 300 requests per minute. Per client app: according to its plan. Login and password reset: 5 per minute per account and per IP. Payments: 10 per minute per user.

How it works, step by step

1. Identifies the caller by IP, client ID and, once known, user ID.
2. Increments the counter for the current window in the shared store (arrow F).
3. If under the limit: passes the request on and adds headers showing remaining quota.
4. If over: returns 429 with a `Retry-After` header and logs the event.

5. If the counter store is unavailable: falls back to a safe local limit rather than allowing unlimited traffic.

3. Authentication (JWT validation)

What it is	Middleware that verifies the access token on every request.
Simple analogy	The guard checking that the badge is genuine, issued by our badge office, for this building, and not expired.
Protects against	Identification and Authentication Failures: forged, expired, altered or wrong-audience tokens.
Checks performed	Token present and well-formed; algorithm is on the allowlist (for example RS256 or ES256; never `none`); signature valid using the key from JWKS (arrow A); `iss` is our provider; `aud` is this API; `exp` not passed and `nbf` reached, with small clock tolerance.

How it works, step by step

1. Reads the `Authorization: Bearer` header; missing token returns 401.
2. Decodes the header and finds the key ID (`kid`).
3. Gets the matching public key from the cached JWKS; unknown key IDs trigger one controlled refresh.
4. Verifies the signature and every required claim; any failure returns 401 with no detail about which check failed.
5. Builds a verified identity object (subject, scopes, roles, tenant) for the next steps (arrow 4).

4. Authorization

What it is	The decision whether this identity may perform this action on this specific resource.
Simple analogy	The badge only opens certain rooms, and inside, only your own drawer.
Protects against	Broken Access Control, the number one OWASP risk, including BOLA/IDOR (other users' records), broken function-level authorization (normal users calling admin endpoints) and mass assignment of privileged fields.
Problem before	Most high-severity findings were here: endpoints that checked login but not ownership, and admin routes protected only by being "hidden".
Change made	Deny by default. Every endpoint declares its required scope or role. Object-level checks are done centrally and repeated in queries. Tenant ID always comes from the token, never from the request.

How it works, step by step

1. Looks up the rule for this endpoint and method (arrow B); endpoints without a rule are denied.
2. Checks the token's scopes and roles against the rule; failure returns 403.
3. For endpoints with an ID (for example `/orders/42`), loads ownership information and confirms the caller owns or may access record 42; failure returns 404 (to avoid confirming the record exists) or 403 by policy.
4. Ignores any user ID, tenant ID or role values sent in the request body; uses only those from the verified token.
5. Logs denied attempts (arrow M); many denials from one user trigger an alert.

5. Input validation

What it is	Automatic checking of every input against the endpoint's OpenAPI schema (arrow C), plus business rules.
Simple analogy	A clerk who rejects any form not filled in exactly according to the template.
Protects against	Injection (SQL, NoSQL, command), mass assignment, SSRF through user-supplied URLs, crashes from unexpected types, and oversized data.
Rules applied	Correct types; required fields present; string length limits; number ranges; formats (UUID, email, date); enums for fixed values; unknown fields rejected; array size limits; URLs only to allowlisted hosts; file uploads checked for type and size.

How it works, step by step

1. Parses the JSON body safely (depth and size limits).

2. Validates path parameters, query parameters, headers and body against the schema.
3. Rejects the request with 400 and a clear, non-sensitive list of field errors if anything fails.
4. Converts validated input into typed objects, so business logic never handles raw strings.
5. Passes valid requests on (arrow 6).

6. Idempotency check

What it is	A guard for operations that create or change things, such as `POST /payments` or `POST /orders`.
Simple analogy	A receipt number that prevents the same payment slip being processed twice.
Protects against	Double charges and duplicate records from network retries, impatient double-clicks, or replayed requests.
Rules	Required header `Idempotency-Key` (a UUID) on unsafe operations. Keys are scoped per user. The store keeps a fingerprint (hash) of the request body. Keys expire after, for example, 24 hours.

How it works, step by step

1. Reads the key; missing key on a required endpoint returns 400.
2. Looks up the key for this user in the idempotency store (arrow D).
3. **New key:** atomically reserves it as “in progress” with the request fingerprint, then passes the request to business logic.
4. **Same key, finished:** returns the saved response without doing the operation again.
5. **Same key, still in progress:** returns 409 (or waits briefly) so two copies never run at once.
6. **Same key, different body:** returns 422/409, because the client is misusing the key.
7. After business logic completes, saves the response under the key.

7. Business logic + data access

What it is	The service code that actually does the work and talks to the database and other APIs.
Simple analogy	The office staff who process the request once it has passed reception.
Protects against	Injection (parameterized queries and ORM only), Cryptographic Failures (encryption of sensitive fields), Security Misconfiguration (safe errors), SSRF (outbound allowlist).
Practices	Parameterized queries everywhere; ownership conditions in queries; least-privilege database accounts per service; secrets from the manager (arrow E); generic error messages to clients with details only in logs; responses return only fields the caller may see.

How it works, step by step

1. Receives a typed, validated, authorised request.
2. Runs the operation using a database connection created with credentials from the secrets manager.
3. Uses parameterized queries that include the owner or tenant condition.
4. Filters the response to allowed fields (no internal IDs, password hashes or other users' data).
5. On errors, returns a generic message with the request ID; full details go to logs.

5.3 Supporting security services

G. Security logging

What it is	Central, structured logging of security events with alerting.
Simple analogy	The building's CCTV and visitor log.
Protects against	Security Logging and Monitoring Failures: attacks going unnoticed.
What is logged	Authentication failures, authorization denials, validation failures, rate-limit hits, idempotency conflicts, admin actions; with request ID, user/client ID, IP, endpoint and outcome. Never tokens, passwords or secrets.

How it works, step by step

1. Each stage emits a structured event (arrow M).
2. Events are shipped to a central log store with retention and tamper protection.
3. Rules raise alerts, for example many 403s from one user (possible ID guessing) or a spike of 401s (credential stuffing).
4. Logs support incident investigation and audit.

F. Rate-limit counters

What it is	A fast shared store, such as Redis, holding rate-limit counters.
Why shared	With many API instances behind a load balancer, local counters would let attackers multiply their limit by the number of instances.

How it works, step by step

1. Receives an atomic increment for the client and window key.
2. Returns the current count; counters expire automatically when the window ends.
3. Is replicated for availability; the limiter has a safe fallback if it is unreachable.

A. JWKS public keys

What it is	A local cache of the identity provider's public signing keys.
Simple analogy	The guard's reference copy of what a genuine badge stamp looks like.
Key details	Only public keys are here, so leaking them is harmless. The private signing key stays inside the identity provider.

How it works, step by step

1. Fetches the JWKS from the provider at start-up (arrow K).
2. Refreshes periodically, and once on an unknown key ID, with rate limiting so attackers cannot force constant refreshes.
3. Keeps old keys until tokens signed with them have expired, allowing smooth key rotation.

B. Permission rules

What it is	Declarative rules: which scope or role each endpoint needs, and how record ownership is decided.
Simple analogy	The access-control list for every room in the building.
Key details	Stored as code next to the endpoints and reviewed like code; tests prove that each role can and cannot do the right things.

How it works, step by step

1. Each endpoint declares, for example, `orders:read` for `GET /orders/{id}` and ownership by `order.customer_id == token.sub`.
2. Admin endpoints require an admin role and are also network-restricted.

3. The authorization stage evaluates these rules for every request.

C. API schema (OpenAPI)

What it is	The machine-readable description of every endpoint: parameters, body fields, types, formats and limits.
Simple analogy	The official template for every form.
Key details	One source of truth used for validation at runtime, documentation, client generation and API security tests.

How it works, step by step

1. Developers update the schema with every API change; CI checks the schema is valid and complete (every field has a type and limit).
2. The validation stage loads it at start-up and checks requests against it.
3. Strict mode: `additionalProperties: false` rejects unknown fields.

D. Idempotency store

What it is	A database table or key-value store holding idempotency records.
Record fields	User ID + key, request fingerprint, status (in progress or done), saved response code and body, expiry time.

How it works, step by step

1. Supports an atomic “insert if not exists” so two simultaneous requests cannot both reserve the same key.
2. Stores the final response when the operation completes.
3. Deletes expired records automatically.

E. Secrets manager

What it is	An encrypted vault that stores and delivers secrets to services at runtime.
Simple analogy	A locked safe whose combination changes regularly, opened only by authorised staff, with every opening recorded.
Protects against	Secrets in source code, config files, container images or logs (Cryptographic Failures and Security Misconfiguration).
Problem before	Database passwords and API keys were in config files and environment files committed to repositories; some were never rotated.
Change made	All secrets moved to the manager; found secrets were rotated (a leaked secret must be considered compromised); services authenticate to the manager with their workload identity; automatic rotation; short-lived dynamic database credentials where supported.

How it works, step by step

1. A service starts and proves its identity to the secrets manager (for example via its cloud or Kubernetes identity).
2. The manager checks its policy: this service may read only its own secrets.
3. The secret is delivered into memory (not written to disk or logs).
4. When the secret rotates, the service picks up the new value without a redeploy.
5. Every access is logged for audit.

Database + downstream APIs

What it is	Where data is stored and other services the API calls.
Hardening	Each service has its own database account with only the permissions it needs; connections use TLS; outbound calls go only to allowlisted hosts with service tokens (client credentials).

How it works, step by step

1. Receives parameterized queries only.
2. Rejects anything the service account is not allowed to do.
3. Downstream APIs validate the service's token in the same way as stage 3.

5.4 CI/CD security gates (Figure 2)

SAST gate (stage 3)

What it is	A static analysis tool (for example Semgrep, CodeQL or SonarQube) run on every pull request.
Simple analogy	An inspector reading the building plans for dangerous designs.
Finds	SQL built from strings, missing authorization annotations on endpoints, disabled TLS verification, weak cryptography, unsafe deserialization, hard-coded credentials, dangerous use of user-supplied URLs.
Tuning	Custom rules added for the company's own frameworks (for example "every controller method must declare a required scope"), and noisy rules tuned so developers trust the results.

How it works, step by step

1. Runs on the changed code (fast) and on the whole codebase nightly (complete).
2. Reports findings in the pull request, next to the line of code.
3. Sends results to the policy decision (stage 6).

Dependency gate (stage 4)

What it is	Software composition analysis (for example Dependabot, Snyk, OWASP Dependency-Check or Trivy).
Simple analogy	Checking every delivered material against a recall list.
Finds	Libraries with known CVEs, including indirect (transitive) dependencies; abandoned libraries; licence problems.

How it works, step by step

1. Builds the SBOM: every library and exact version.
2. Compares it with vulnerability databases and checks whether a fixed version exists.
3. Automatically opens upgrade pull requests for fixable issues.
4. Sends findings to the policy decision; it also rescans existing services daily, because new CVEs are published for old libraries.

Secret-scan gate (stage 5)

What it is	A scanner (for example Gitleaks or built-in repository secret scanning) that looks for credentials in code and history.
Simple analogy	Checking that nobody left a key taped to the plans.

How it works, step by step

1. Runs as a pre-commit hook on developers' machines and again in CI.
2. Detects key formats and high-entropy strings.
3. Fails the build; the secret is rotated in the secrets manager, because once committed it must be treated as leaked.

Policy decision (stage 6)

What it is	The rule engine that turns scan results into pass or fail.
Rules	Fail on any new High or Critical finding. Medium findings create tickets with deadlines. Waivers (accepted risks) need security approval, a reason and an expiry date.
Why it matters	This is what stops the finding count from growing again: new serious problems cannot be merged.

How it works, step by step

1. Collects results from SAST, dependency and secret scans.
2. Removes findings covered by valid, unexpired waivers.
3. Fails the pipeline if any High or Critical remain, posting a summary in the pull request (red feedback line).
4. Passes the pipeline otherwise, recording results for metrics.

Review, staging, API security tests and production (stages 7–10)

Review + merge	A second engineer reviews the change, with a security checklist for endpoints (auth, ownership, validation, idempotency).
Signed artifact	The build is signed so production only runs artifacts produced by the trusted pipeline (Software and Data Integrity).
API security tests	DAST against staging plus automated tests: calls without tokens, with expired tokens, as another user (BOLA), with invalid input, beyond rate limits, and duplicate idempotent requests.

How it works, step by step

1. Merge creates a signed artifact.
2. Staging deploys it with secrets injected from the manager.
3. API security tests run; failures block promotion.
4. Production rollout is gradual and monitored using the security logs.

6. Workflows Explained End to End

6.1 Getting a token (OAuth2)

User-facing app (authorization code with PKCE):

1. The app creates a random secret (code verifier) and its hash (code challenge).
2. It redirects the user to the identity provider's login page with the challenge and the requested scopes.
3. The user logs in (password, MFA). The provider returns a one-time authorization code to the app.
4. The app exchanges the code plus the original verifier for tokens; the provider checks the verifier matches the challenge.
5. The app receives a short-lived access token (JWT) and a refresh token.
6. When the access token expires, the app uses the refresh token to get a new one; refresh tokens rotate on each use.

Service-to-service (client credentials): the calling service authenticates to the provider with its client secret (stored in the secrets manager) or a certificate, and receives an access token with only the scopes it needs.

6.2 A normal request, end to end

Step	Where	What happens, in simple words
T	Client ↔ Identity provider	The client already holds a valid access token.
1	Gateway	`POST /payments` arrives over HTTPS with the token and an Idempotency-Key. Size and route checks pass.
2	Rate limiter	The user's counter for payments is 3 of 10 this minute: allowed.
3	Authentication	Signature, issuer, audience and expiry are valid. Identity: user 123, scope `payments:write`.
4	Authorization	The endpoint needs `payments:write`: present. The source account in the request belongs to user 123: allowed.
5	Validation	Amount is a positive decimal within limits, currency is in the allowed list, no unknown fields: valid.
6	Idempotency	The key is new for user 123: reserved as "in progress".
7	Business logic	Payment created using parameterized queries and credentials from the secrets manager.

Step	Where	What happens, in simple words
8	Database	Data stored under a least-privilege account.
6b	Idempotency	Response saved under the key.
–	Response	201 Created returned to the client; events logged (arrow M).

6.3 Rejection paths (where bad requests stop)

Situation	Stopped at	Response	Logged as
Too many requests or login attempts	2. Rate limiter	429 + Retry-After	Rate-limit hit
Missing, forged, expired or wrong-audience token	3. Authentication	401	Auth failure
Valid user calls an admin endpoint	4. Authorization	403	Access denied (function)
Valid user requests someone else's order	4. Authorization	404 (or 403)	Access denied (object), alert if repeated
Extra field like "role": "admin"	5. Validation	400	Validation failure
SQL fragment in a name field	5. Validation (and parameterized queries)	400	Validation failure
Same payment retried after a timeout	6. Idempotency	Original 201 replayed	Idempotent replay
Same key with a different amount	6. Idempotency	422 / 409	Idempotency conflict
Unexpected server error	7. Business logic	500 with request ID only	Error with full details

6.4 Rate limiting workflow

1. Build counter keys for this request: per IP, per client app, per user, and per sensitive endpoint.
2. Atomically increment each counter in the shared store for the current window.
3. Compare with the configured limits; the strictest limit wins.
4. Under all limits: continue, adding remaining-quota headers.

5. Over any limit: return 429 with Retry-After; log it; repeated abuse can trigger a temporary block.
6. For login: after several failures for one account, add delays or require extra verification, while avoiding locking out the real user permanently.

6.5 Idempotent operation workflow

1. The client generates a UUID as the Idempotency-Key for this one payment and sends the request.
2. The server reserves the key with the request fingerprint (atomic insert).
3. The payment is processed; the response is saved under the key.
4. The network drops before the client gets the answer; the client retries with the **same** key.
5. The server finds the finished record with the same fingerprint and returns the saved response. No second payment.
6. If the retry arrives while the first is still running, the server returns 409 (or waits), so the payment never runs twice in parallel.
7. After 24 hours the record expires.

6.6 Secrets lifecycle workflow

1. **Discover:** secret scanning finds credentials in repositories and config files.
2. **Rotate:** every discovered secret is changed at its source, because it may already be compromised.
3. **Store:** the new secret goes only into the secrets manager, with a policy naming which service may read it.
4. **Deliver:** at start-up, each service proves its identity and receives its secrets in memory.
5. **Rotate regularly:** the manager changes secrets automatically (or issues short-lived dynamic credentials); services reload them without downtime.
6. **Audit:** every read is logged; unusual access triggers an alert.
7. **Revoke:** if a leak is suspected, the secret is revoked immediately and a new one issued.

6.7 Signing-key rotation workflow (JWT)

1. The identity provider creates a new signing key pair and publishes the new public key in the JWKS alongside the old one.
2. APIs refresh their JWKS cache and now know both keys.
3. The provider starts signing new tokens with the new key (new `kid`).
4. Tokens signed with the old key keep working until they expire (minutes).
5. The old public key is removed from the JWKS after all old tokens have expired.

6.8 CI pipeline workflow

1. A pre-commit hook on the developer's machine runs a quick secret scan before code leaves the laptop.

2. The developer opens a pull request (stage 1); the build and unit tests run (stage 2).
3. SAST, dependency and secret scans run in parallel (stages 3–5).
4. The policy decision (stage 6) fails the pipeline on any unwaived High or Critical finding and posts results in the pull request.
5. The developer fixes the code or upgrades the library and pushes again; the pipeline reruns.
6. When everything passes, a reviewer approves and merges (stage 7); a signed artifact is produced.
7. The artifact deploys to staging (stage 8); API security tests run (stage 9).
8. If they pass, the same artifact rolls out to production gradually (stage 10).

6.9 Vulnerability management workflow (fixing what already existed)

1. **Baseline:** run all scanners across every API repository and collect pentest and bug-report findings into one tracker.
2. **Deduplicate and rate:** merge duplicates; rate each by severity (CVSS) and real exploitability.
3. **Group by root cause:** for example, “no object-level checks” appears in 12 endpoints; fixing it once in shared middleware closes all 12.
4. **Set deadlines:** for example Critical within 7 days, High within 30 days.
5. **Fix:** apply the shared controls in sections 5.2 and 5.3, upgrade libraries, rotate secrets.
6. **Verify:** re-scan and re-test to confirm each finding is closed.
7. **Prevent:** CI gates stop new High/Critical findings; daily dependency rescans catch newly published CVEs.
8. **Measure:** track open High/Critical findings over time; compare comparable periods to show the 50% reduction.

7. How the Controls Map to the OWASP Top 10

The table uses the OWASP Top 10 (2021) categories and adds the closely related API Security Top 10 items where they fit.

OWASP risk	What it means simply	Controls in this project
A01 Broken Access Control (incl. API BOLA, function-level auth)	Users can do or see things they should not.	Deny-by-default authorization, scope and role checks, object ownership checks in middleware and queries, tenant from token only.
A02 Cryptographic Failures	Sensitive data or secrets exposed by weak or missing encryption.	HTTPS only, HSTS, secrets manager, no secrets in code or logs, strong JWT algorithms.
A03 Injection	Input treated as code.	Schema validation with allowlists, typed input, parameterized queries, SAST rules for string-built queries.
A04 Insecure Design	Missing protections in the design itself.	Idempotency for money-moving operations, rate limits on sensitive flows, threat modelling of new endpoints.
A05 Security Misconfiguration	Unsafe defaults, verbose errors, open CORS.	Gateway hardening, strict CORS, security headers, generic error messages, admin routes not exposed.
A06 Vulnerable and Outdated Components	Libraries with known holes.	Dependency scanning gate, SBOM, automatic upgrade PRs, daily rescans.
A07 Identification and Authentication Failures	Weak login or token handling.	OAuth2 with PKCE, short-lived JWTs, full claim validation, MFA, rate-limited login.
A08 Software and Data Integrity Failures	Untrusted code or updates reaching production.	CI gates, code review, signed artifacts, same artifact promoted from staging to production.
A09 Security Logging and Monitoring Failures	Attacks go unnoticed.	Structured security events from every stage, central logs, alerts on suspicious patterns.
A10 Server-Side Request Forgery	The server is tricked into calling internal addresses.	Validation of user-supplied URLs against an allowlist; outbound egress restrictions.

OWASP risk	What it means simply	Controls in this project
API: Unrestricted Resource Consumption	Clients exhaust resources or money.	Rate limiting, request size and pagination limits.

8. Where the 50% Came From

The table is an **illustrative** example of open High/Critical findings by category, comparing equivalent review periods before and after the project.

Category	Before	After	Main fixes
Broken access control (BOLA, missing role checks)	12	5	Central authorization middleware, ownership in queries
Vulnerable dependencies	10	6	Dependency gate, upgrades, daily rescans
Injection and unsafe input handling	6	2	Schema validation, parameterized queries, SAST rules
Exposed secrets	5	1	Secrets manager, rotation, secret-scan gate
Security misconfiguration	4	3	Gateway hardening, safe errors, strict CORS
Authentication weaknesses	3	3	OAuth2/JWT validation (remaining items scheduled)
Total High / Critical	40	20	About 50% lower

Why the reduction lasts

Most fixes were made once in shared layers (gateway, middleware, schema validation, secrets manager), so every endpoint benefits automatically, including new ones. The CI gates stop new High and Critical problems from being merged, so the count does not creep back up.

9. Quick Recap

Question	Short answer
What was achieved?	About 50% fewer High/Critical security findings across the REST APIs.
What protects each request?	Gateway hardening, rate limiting, JWT authentication, authorization with ownership checks, schema input validation and idempotency, in that order, before business logic.
How do users and services prove identity?	OAuth2: authorization code with PKCE for apps, client credentials for services; short-lived signed JWTs verified with JWKS keys.
Where do secrets live?	Only in a secrets manager, delivered at runtime, rotated, audited; never in code.
How is new risk kept out?	SAST, dependency and secret-scan gates in CI with a fail-on-High policy, plus API security tests in staging.
Which OWASP risks were covered?	All ten categories, with the biggest gains in broken access control, vulnerable components, injection and exposed secrets.